Premise Chain Guided Retrieval of Relevant Historical Fixes for Automated Vulnerability Repair

Shunyu Liu, Mark Utting, Guowei Yang

School of Electrical Engineering and Computer Science,
The University of Queensland, Brisbane, QLD, 4072, Australia

Abstract

Automated vulnerability repair (AVR) is vital for scaling software security. Recent work has shifted from traditional AVR techniques to LLM-based approaches, which show promising potential in generating security patches. Some LLM-based repair methods further retrieve historical fixes as additional guidance for patch generation. However, we observe a practical limitation in such retrieval-based settings: difficulty in retrieving truly relevant historical fixes when vulnerable functions are long and contain substantial irrelevant code patterns, where surface-level similarity can mislead retrieval and dilute repair guidance. To address these limitations, we propose PCRepair, a novel AVR approach that first introduces premise chain, a novel code representation and then leverages it to retrieve relevant historical fixes for vulnerability repair. The premise chain captures vulnerability-relevant semantics by organizing program logic into semantically coherent blocks connected through explicit dependency relations, while filtering out irrelevant code patterns. The matched relevant historical fixes are incorporated into the LLM's input prompt, guiding the model to generate repairs.

We evaluated PCRepair on the SecVulEval and PrimeVul benchmarks. PCRepair significantly outperformed all baselines, achieving CodeBLEU scores of 52.49 (+32.8% improvement) on SecVulEval and 52.04 (+11.5%) on PrimeVul. These results demonstrate that the premise chain transformation effectively captures vulnerability-relevant semantics and improves the accuracy of automated vulnerability repair.

Keywords: Vulnerability Repair, Large Language Model

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

1. Introduction

Software vulnerabilities pose serious threats to modern software systems, as attackers can exploit them to violate fundamental security properties such as confidentiality, integrity, and availability [28, 16]. Recent advances in vulnerability detection [26] techniques have further accelerated the identification and reporting of these issues. For example, industry reports show that a large majority of software projects contain at least one open-source vulnerability [25], and the number of publicly disclosed CVEs continues to rise each year. However, the processes of vulnerability triage and remediation have not kept pace with this growth. A substantial portion of reported vulnerabilities remain unverified for extended periods, and even confirmed issues often take weeks or months to be patched [10, 8]. Moreover, vulnerability repair remains a highly labor-intensive task [6]. Together, these challenges highlight the urgent need for scalable and efficient approaches to support timely and reliable vulnerability remediation.

To alleviate this challenge, automatic vulnerability repair (AVR) has emerged as a promising research direction. Prior studies on AVR have explored a diverse set of methodologies, which can generally be grouped into four categories: template-guided [15], search-based [31], constraint-based [12] and learning-based [11, 32]. Recent work has shifted from traditional AVR to LLM-based approaches. Methods such as Vul-R2 [27] leverage large language models to incorporate reasoning capabilities into the repair process.

Recent AVR approaches have shown that historical fixes can provide useful guidance for vulnerability repair. For example, VulMaster [32] and APPATCH [19] both incorporate similar fixed examples into their repair process, highlighting the value of historical fixes in improving patch generation. However, the effectiveness of such guidance depends critically on whether the retrieved fixes are truly relevant to the target vulnerability. *In practice, identifying such fixes remains challenging.* Vulnerable functions often contain substantial irrelevant code context, which can obscure the core vulnerability logic and cause retrieval to focus on superficial resemblance, such as local syntax, variable usage, or implementation details, rather than the underlying repair intent. As a result, the retrieved fixes may appear similar at the code level, yet provide limited guidance for accurate patch generation.

Our work. To address the above challenges, we propose **PCRepair**, an automated vulnerability repair framework built on a new code representation called the premise chain. premise chain reformulates code into a set of se-

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

mantically coherent blocks connected by explicit premise relations. Inspired by the PARC representation for analyzing reasoning chains in mathematical problem solving [17], premise chain adapts this idea to program logic by combining abstraction and causality: it abstracts low-level statements into higher-level logical actions and links these actions through logical prerequisites rather than textual or execution order. Based on this representation, we perform PCRepair. This comparison retrieves historical cases whose vulnerable logic is semantically aligned with the target. The corresponding fixes of the retrieved cases are then used as guidance for LLM-based patch generation. In this way, PCRepair uses premise chain to reduce irrelevant implementation details, retrieve more relevant historical repair knowledge, and provide more useful context for vulnerability repair.

Built on this idea, PCRepair reformulates vulnerability repair as a process of representation, matching, and generation. Given a target vulnerable function, PCRepair first constructs its premise chain representation, where an LLM organizes the raw code into semantically coherent blocks linked by explicit premise relations. Based on this representation, PCRepair then retrieves historical fixes whose repair logic is most relevant to the target vulnerability through premise chain matching, rather than relying on surface code similarity alone. Finally, the retrieved historical fixes are incorporated as guidance for patch generation, enabling the model to produce the final repair.

To evaluate the effectiveness of PCRepair, we compare it with state-of-the-art (SOTA) vulnerability repair baselines on two widely used C/C++ benchmarks, SecVulEval [1] and PrimeVul [7]. Experimental results show that PCRepair achieves the best performance across both datasets in terms of CodeBLEU and Exact Match (EM). On SecVulEval, PCRepair improves over the strongest supervised baseline by 32.8% in CodeBLEU and 75.3% in EM. On PrimeVul, PCRepair again attains the highest scores (CodeBLEU 52.04, EM 27.02), outperforming all AVR baselines.

In summary, this paper makes the following contributions:

- We propose the premise chain, a novel code representation designed to capture both abstraction and causality. This representation helps capture the root cause of a vulnerability while filtering out irrelevant structural noise.
- We introduce PCRepair, an AVR framework that leverages premise chain to retrieve historical fixes relevant to the target vulnerability and

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

incorporates these fixes to guide LLM-based vulnerability repair.

- We implement and evaluate PCRepair [2] on two C/C++ benchmarks, SecVulEval and PrimeVul, in comparison to the state-of-the-art AVR approaches. The experimental results show that PCRepair achieves the best performance across both datasets, demonstrating the effectiveness of premise chain in identifying relevant historical fixes for LLM-based vulnerability repair.

2. Premise Chain

2.1. Overview of the Premise Chain

A premise chain is a structured code representation that reformulates a function into a set of semantically coherent blocks connected by explicit premise relations. Each block summarizes one complete logical action implemented by one or more source-code statements, while each premise relation indicates that one block provides the necessary condition or supporting context for another block. Compared with a flat sequence of code statements, premise chain makes the high-level program logic easier to inspect by organizing code according to semantic actions and their dependencies.

Formally, a premise chain is an ordered set of blocks $B = \{b_1, b_2, \ldots, b_n\}$. Each block b_i is represented as a tuple (id, D, L, P), where:

- **Block ID (id):** a unique identifier that distinguishes the block from others in the chain;
- **Description (D):** a concise semantic summary describing the purpose or role of the code captured by the block;
- **Line Numbers (L):** the corresponding source-code line number(s) to which the block is grounded;
- **Premises (P):** a set of references to preceding blocks that provide the logical conditions or supporting context for the current block.

Fig. 1 illustrates how premise chain represents vulnerable functions and supports the retrieval of relevant historical fixes. The upper-left subfigure shows the target vulnerable function, where the green highlighted line indicates the human-written fix for illustration. The upper-right subfigure

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

presents the corresponding premise chain representation, which abstracts the function into semantically coherent blocks with explicit premise relations. The lower-left subfigure shows a fixed vulnerability function retrieved by premise chain matching, and the lower-right subfigure shows its premise chain representation. The red boxes and arrow highlight the matched vulnerability-relevant logic between the target and retrieved functions.

This example of Fig. 1 illustrates the two key benefits of premise chain: *abstraction* and *causality*.

- First, premise chain provides abstraction by grouping low-level code statements into semantically coherent blocks. Instead of treating each statement separately, premise chain summarizes a group of related statements as one complete logical action. For example, the statements that parse the route endpoints and construct the route key can be represented as higher-level semantic blocks, rather than as isolated calls to attribute lookup, JID parsing, or string construction. This abstraction reduces the influence of superficial implementation details and makes it easier to compare the high-level logic of different functions.

- Second, premise chain captures causality by explicitly modeling premise relations between blocks. These premise relations indicate which blocks provide the necessary logical support for later blocks. This is different from simply following textual order or execution order. In Fig. 1, Block 1 appears before Block 2 in the original code, but it is not a premise of Block 2 because extracting the optional `id` attribute is not required for the route-state update. Instead, Block 2 depends on Block 0, which provides the route-related information needed to decide and perform the state update. This shows that premise chain captures logical dependency rather than merely preserving statement order.

The bottom part of Fig. 1 further illustrates how these two properties affect the comparison between different functions. In the premise chain representation, the target vulnerability and the fixed vulnerability retrieved by premise chain matching contain semantically similar state-update blocks. The block highlighted by the red box in the target vulnerable function can be summarized as:

> *When the result message is marked as valid, update the route state associated with the route key to valid.*

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

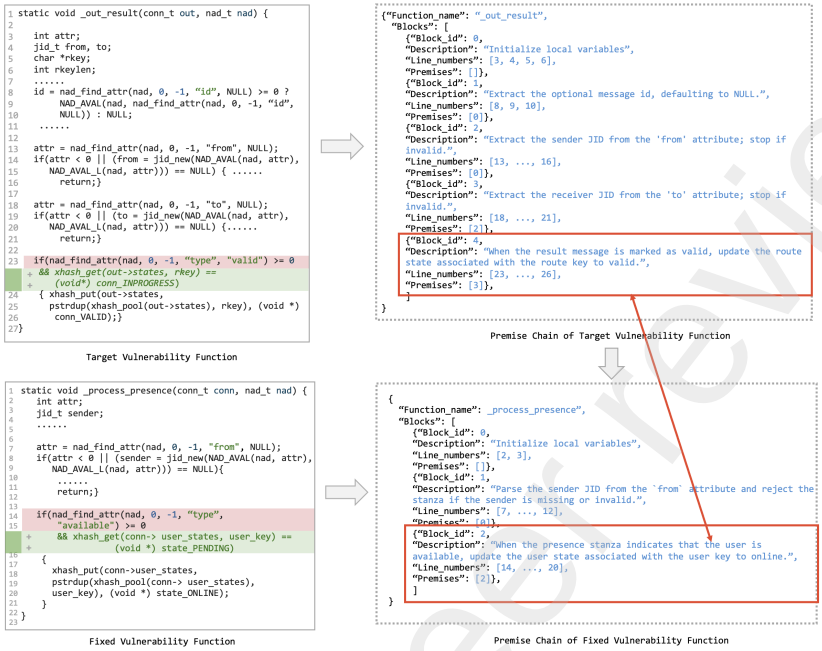


Figure 1: Example of a historical vulnerability fix retrieved using premise chain matching.

The corresponding block highlighted by the red box in the fixed vulnerability can be summarized as:

> *When the presence stanza indicates that the user is available, update the user state associated with the user key to online.*

Although the two functions use different variables and operate on different states, these two blocks express the same high-level behavior: an incoming message condition directly triggers a key-associated state update. This similarity becomes clear in the premise chain format because the relevant behavior is abstracted into a semantic block, while unrelated surrounding statements are separated into other blocks through premise relations.

A straightforward alternative is to retrieve historical fixed vulnerabilities using control flow information. In control flow matching, we trace backward from the target branch along the execution order and use the resulting statements to match similar fixed vulnerabilities. As shown by the yellow-highlighted statements in Fig. 2, this traceback includes preceding statements such as variable declarations, attribute parsing, and optional identifier extraction. However, the fixed vulnerability retrieved by control flow matching

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

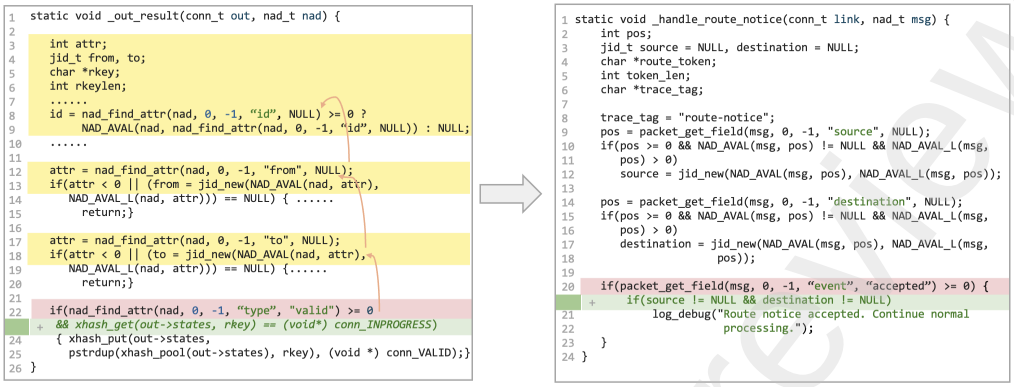


Figure 2: Example of a historical vulnerability fix retrieved using control flow matching.

is mainly similar in terms of the surrounding control flow structure. These statements do not necessarily capture the same core behavior. In particular, the retrieved fixed vulnerability checks whether parsed identifiers are non-null before continuing the processing, whereas the target premise chain block describes a key-associated state update triggered by an incoming message condition. Therefore, although the retrieved fixed vulnerability appears structurally similar, it is less aligned with the core logic captured by the target premise chain block.

This comparison shows why abstraction and causality are important. Abstraction makes the relevant behavior comparable at the level of semantic actions, while causality prevents unrelated preceding statements from being treated as necessary context. As a result, premise chain can expose semantically similar logic that may be obscured when using control flow alone.

2.2. Premise Chain Construction

To construct a premise chain, we prompt an LLM to analyze the input function and reformulate it into a sequence of structured reasoning blocks. The LLM is instructed to identify semantically coherent code regions, summarize each region as one complete logical action, ground the block in the corresponding source-code line number(s), and specify the necessary premise blocks that support the current block. The prompt is shown below:

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

[Task Description] Please decompose the given function into a reasoning premise chain, represented as a sequence of structured steps. Each step must:

1. Describe one clear and complete logical action (not a syntax fragment),
2. Include the exact code line number(s) involved, and
3. Specify its supporting premises by referencing only the necessary previous steps.

Your output must strictly follow the format of the provided premise chain example.

1. This is the function you need to decompose: *<Target vulnerable code>*
2. This is one output example: *<A generic premise chain example>*

3. PCRepair

3.1. Overview

After defining the premise chain representation, we now describe how PCRepair uses premise chain to support vulnerability repair. Given a target vulnerable function and a collection of historical vulnerability-fix pairs, the goal of PCRepair is to retrieve historical cases whose vulnerable-side logic is semantically aligned with the target vulnerability, and then use their corresponding fixes as guidance for LLM-based patch generation.

Importantly, the retrieval process is performed on vulnerable functions rather than by directly comparing patches. Specifically, PCRepair compares the premise chain representations of the target vulnerable function and historical vulnerable functions to identify cases with similar vulnerability-side logic. The fixes associated with the top-ranked historical cases are introduced only after retrieval, when constructing the prompt for patch generation. This design allows PCRepair to exploit useful historical repair knowledge while avoiding direct patch-level matching.

Fig. 3 presents an overview of the proposed PCRepair framework. PCRepair first performs *Vulnerability Decomposition via premise chain*, where an LLM constructs the premise chain representation of the target vulnerable

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

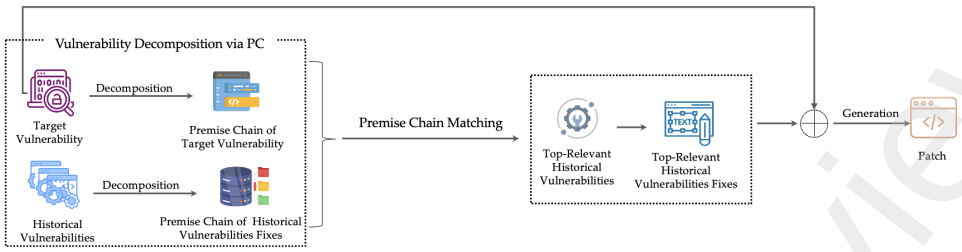


Figure 3: Overall Framework of PCRepair

function. For historical vulnerabilities, PCRepair first restricts retrieval to those with the same CWE type as the target vulnerability and applies a Cost-Aware Coarse Filtering step for efficiency. It then constructs premise chain representations only for the retained historical vulnerabilities and performs premise chain matching. Next, it performs *Premise Chain Matching* to identify historical vulnerabilities whose repair logic is most relevant to the target vulnerability within the same CWE category. In this stage, *Multi-Source Block Identification* starts from the code regions directly modified in historical vulnerabilities and traces their premise relations within the premise chain to identify compact contexts that are relevant to the vulnerability. Based on these identified contexts, *Importance-Aware Block Matching* matches the target vulnerability with historical vulnerabilities by jointly considering semantic similarity and block importance, and ranks the historical vulnerabilities accordingly. Finally, in the *Patch Generation* stage, the target vulnerable code together with the selected reference pairs is incorporated into the prompt to guide the LLM in generating the final patch.

3.2. Premise Chain Matching

3.2.1. Cost-Aware Coarse Filtering

To find suitable historical fixes for repair guidance, PCRepair first retrieves historical fixes that share the same CWE type as the target vulnerability. This design is motivated by the observation that vulnerabilities within the same CWE category often share similar root causes and repair patterns, making their fixes more transferable across different codebases. Restricting retrieval to historical fixes with the same CWE type therefore provides a semantically meaningful starting point for example retrieval.

However, even within a single CWE category, the number of historical vulnerabilities can be substantial. Applying premise chain construction to

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

all historical fixes would incur prohibitive computational cost. To address this issue, PCRepair introduces a cost-aware coarse filtering stage that efficiently narrows down the historical fixes before performing deeper structural analysis. Instead of performing expensive analysis on all historical fixes from the same CWE category, this stage serves as a low-cost quick filter that leverages lightweight similarity signals to approximate relevance. Specifically, we employ: (1) semantic similarity based on CodeBERT embeddings, (2) structural proximity captured by file-path matching, and (3) dataflow-aware similarity approximated by CodeBLEU scores. According to the aggregated coarse relevance score, only the top 10% of historical fixes are preserved for the subsequent premise chain matching, substantially reducing the overall computational overhead. We define a coarse relevance score that aggregates the above similarity measures into a single ranking criterion:

$$S(v_t, v_i) = \alpha \cdot \mathrm{Sim}_{\mathrm{sem}}(v_t, v_i) + \beta \cdot \mathrm{Sim}_{\mathrm{path}}(v_t, v_i) + \gamma \cdot \mathrm{Sim}_{\mathrm{df}}(v_t, v_i), \qquad (1)$$

where v_t denotes the target vulnerability and v_i represents historical fixed vulnerabilities from the same CWE-category. $\mathrm{Sim}_{\mathrm{sem}}$, $\mathrm{Sim}_{\mathrm{path}}$, and $\mathrm{Sim}_{\mathrm{df}}$ correspond to the semantic, path, and dataflow-aware similarity measures.

For simplicity, we assign equal weights to the three coefficients in all experiments, i.e., $\alpha = \beta = \gamma = \frac{1}{3}$. This gives each component an equal contribution to the final matching score.

3.2.2. Multi-Source Block Identification (MSBI)

After the cost-aware coarse filtering stage, PCRepair applies a Multi-Source Block Identification mechanism to identify the code blocks most relevant to each historical fix and assign them importance scores based on the premise relations encoded in the premise chain.

PCRepair first identifies a set of focal blocks F, i.e., the blocks that directly contain the code lines modified by the patch. These blocks mark where the fix is applied. Starting from each focal block $b_f \in F$, PCRepair recursively traces its premise references and collects all reachable prerequisite blocks. This process yields an expanded block set $\mathcal{B}$, which contains both the patched blocks and the supporting blocks logically connected to them. We treat $\mathcal{B}$ as the repair-relevant block set for the historical fix.

Not all blocks in $\mathcal{B}$ are equally important to the fix. In particular, blocks that are closer to the patched region should receive higher importance, since

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

they are more directly related to the modified code. Moreover, when a fix involves multiple focal blocks, prerequisite blocks shared by several focal chains are also more important, as they capture logical conditions that jointly support multiple modified regions. To reflect these two factors, PCRepair assigns each block $b_i \in \mathcal{B}$ a cumulative influence score based on its premise distance to the focal blocks:

$$W(b_i) = \min\left(1.0, \sum_{b_{fj} \in F} \frac{1}{1 + dist(b_i, b_{fj})}\right), \tag{2}$$

where $dist(b_i, b_{fj})$ denotes the number of premise-tracing steps from b_i to the focal block b_{fj}. Under this formulation, a block receives a larger contribution from focal blocks that are closer to it, and its total score further increases if it appears in the premise chain of multiple focal blocks. These influence scores are then used in the subsequent alignment stage to weight block-level similarity, so that blocks with stronger relevance to the fix contribute more to the final matching score.

3.2.3. Importance-Aware Block Matching (IABM)

After identifying vulnerability-relevant blocks, PCRepair matches the target vulnerability with each historical vulnerability at the premise chain level. The goal is to retrieve historical cases whose vulnerable-side logic is semantically aligned with the target, rather than directly comparing their patches. The corresponding fixes of the top-ranked historical cases are used only after retrieval, as guidance for patch generation.

Each premise chain block is represented by two attributes: a semantic embedding and an influence score. The semantic embedding is used to measure the meaning-level similarity between two blocks, while the influence score serves as an importance weight indicating how central the block is to the vulnerability-relevant logic. During matching, blocks with higher influence scores contribute more to the final score, allowing PCRepair to emphasize core vulnerability-related logic rather than peripheral code.

Given a target vulnerability with blocks $T = \{t_1, t_2, \ldots, t_N\}$ and a historical vulnerability with blocks $U = \{u_1, u_2, \ldots, u_M\}$, IABM computes alignment from two complementary directions: weighted recall and weighted precision. Here, recall and precision are not classification metrics; instead, they describe two directions of block-level alignment.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

Weighted recall measures target coverage: it answers the question, *Does the historical vulnerability cover the important logic of the target vulnerability?* For each target block t_i, PCRepair finds the most similar block in the historical vulnerability and weights this best-match score by the influence score of t_i:

$$S_{\text{recall}} = \frac{\sum_{i=1}^{N} w_i^T \cdot \max_j \text{CosSim}(\mathbf{v}_{t_i}, \mathbf{v}_{u_j})}{\sum_{i=1}^{N} w_i^T}. \tag{3}$$

In this formula, $\max_j \text{CosSim}(\mathbf{v}_{t_i}, \mathbf{v}_{u_j})$ selects the best matching historical block for target block t_i. The weight w_i^T makes important target blocks contribute more to the recall score. Therefore, if a critical target block cannot find a similar block in the historical vulnerability, the recall score decreases substantially.

Weighted precision answers the reverse question: *Is the historical vulnerability focused on the same logic as the target?* For each historical block, PCRepair finds the most similar block in the target vulnerability. These best-match scores are aggregated according to the influence scores of the historical blocks. Thus, if a historical vulnerability contains many important blocks that are unrelated to the target, its precision score will be low.

$$S_{\text{precision}} = \frac{\sum_{j=1}^{M} w_j^U \cdot \max_i \text{CosSim}(\mathbf{v}_{u_j}, \mathbf{v}_{t_i})}{\sum_{j=1}^{M} w_j^U}. \tag{4}$$

This score penalizes historical vulnerabilities that contain important blocks unrelated to the target. In other words, a candidate should not receive a high score merely because it matches a small part of the target while containing other important but irrelevant logic.

Finally, we combine the two directions using their harmonic mean:

$$Score_{\text{final}} = \frac{2 \cdot S_{\text{recall}} \cdot S_{\text{precision}}}{S_{\text{recall}} + S_{\text{precision}}}. \tag{5}$$

A historical case receives a high final score only when it both covers the important blocks of the target and keeps its own important blocks aligned with the target. The final score is used to rank historical vulnerability-fix pairs. After ranking, the fixes associated with the top-ranked historical cases are incorporated into the prompt as guidance for LLM-based patch generation.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

3.3. Patch Generation

In the final stage, PCRepair generates patches by leveraging the relevant historical fixes selected through premise chain matching. For each target vulnerability, the most relevant historical fixes are ranked and incorporated as guidance for patch generation. We employ ChatGPT for patch generation using a structured prompting strategy that provides the model with highly relevant repair context. Specifically, the prompt includes four key components:

- The target vulnerable function,
- The specific code requiring modification,
- The CWE type and its description provided to the model,
- The retrieved relevant historical fixes.

4. Experiment Setup

4.1. Research Questions

We evaluate PCRepair on the following four research questions (RQs).

- **RQ1:** How effective is PCRepair in repairing vulnerabilities?
- **RQ2:** How does the core design of PCRepair contribute to its effectiveness?
- **RQ3:** How does the quality of premise chain construction affect vulnerability repair performance?
- **RQ4:** How do hyper-parameter choices influence the performance of PCRepair?

4.2. Dataset

In this paper, we focus on C/C++ programs, as they are widely used in real-world software systems and are frequently affected by security vulnerabilities that have been carefully documented by security researchers. To evaluate the effectiveness of our approach, we utilize two datasets: SecVulEval [1] and PrimeVul [7].

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

SecVulEval. SecVulEval is a benchmark for fine-grained evaluation of vulnerability analysis techniques on real-world `C/C++` code. Unlike function-level datasets, it provides statement-level vulnerability annotations, enabling more precise assessment of a model's ability to localize and reason about vulnerabilities. The benchmark contains 25,440 function samples covering 5,867 unique CVEs from projects spanning 1999 to 2024.

PrimeVul. PrimeVul is a large-scale, real-world C/C++ vulnerability dataset that aggregates vulnerability-related commits from four established datasets: BigVul [9], CVEFixes [3], CrossVul [18], and DiverseVul [4]. It covers more than 140 CWE categories and applies a stringent deduplication process to reduce redundancy.
shunyuIn addition, we follow the temporally-aware data split provided by PrimeVul, which results in 3,789, 480, and 435 samples for training, validation, and testing, respectively.

We use SecVulEval as the primary dataset in our evaluation because it is more comprehensive than previous datasets, containing a larger number of non-duplicated vulnerability samples and provides richer annotation information, which supports detailed and controlled analysis of PCRepair across multiple research questions. PrimeVul is used as a complementary dataset to facilitate a fair comparison with existing AVR methods that are evaluated on this benchmark. The differences in dataset usage and experimental settings are discussed in detail in RQ1.

4.3. Baselines

We compare PCRepair with three categories of baselines.

- First, we include two SOTA LLM-based AVR approaches, Vul-R2 [27] and APPATCH [19], which represent the most competitive recent methods.
- Second, we consider two widely used AVR baselines, VulRepair [11] and VulMaster [32], which are commonly adopted in prior work for comparison.
- Third, we include two LLMs, ChatGPT-4 [20] and ChatGPT-5 [21].

In addition, since PCRepair is implemented with both GPT-4 and GPT-5, we also report the results of ChatGPT-4 [20] and ChatGPT-5 [21] in our experiments.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

4.4. Evaluation Metrics

Following prior studies [27, 32, 11, 5], we evaluate repair performance using two widely adopted metrics: Exact Match (EM) and CodeBLEU [24]. Exact Match (EM) measures the percentage of generated fixes that are syntactically identical to the ground-truth patches, calculated as the ratio of perfect matches to the total number of evaluated vulnerabilities. To further assess the similarity between generated patches and reference fixes, we adopt CodeBLEU, a code-specific variant of BLEU [22].

4.5. Implementation Details

We implement PCRepair in Python [23]. In the cost-aware coarse filtering stage, we retain the top 10% of historical fixed vulnerabilities for subsequent premise chain decomposition. For patch generation, we use the top two retrieved historical fixes as repair guidance and generate one candidate patch for each vulnerability for evaluation. For premise chain decomposition, we use ChatGPT-5 in all settings. In the generation stage, we use ChatGPT-4 and ChatGPT-5, respectively, as reported in RQ1. We set the sampling temperature to 0.8 [29] to maintain diversity in the generated outputs.

5. Experiment Results

5.1. *RQ1: Effectiveness of PCRepair*

Setup. We evaluate our proposed PCRepair framework against vulnerability repair baselines on two widely used benchmarks, SecVulEval and PrimeVul. We report results using standard evaluation metrics, EM and CodeBLEU, following prior work [11, 32]. Due to the substantial computational cost of training Vul-R2, we report its results only on PrimeVul, using the performance numbers reported in the original Vul-R2 paper.

Results. Table 1 presents the performance comparison between PCRepair and a set of baseline methods on the SecVulEval and PrimeVul benchmarks. The best results are highlighted in bold. Overall, under both GPT-4 and GPT-5, PCRepair achieves the best performance across both datasets and evaluation metrics, demonstrating the effectiveness of PCRepair.

On SecVulEval, PCRepair achieves the highest overall performance with a CodeBLEU of 52.49 and an EM of 26.87. Notably, under a fair comparison using ChatGPT-4, PCRepair decisively outperforms APPATCH, improving CodeBLEU from 43.34 to 50.12 and EM from 20.95 to 24.56. Compared

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

Table 1: Performance Comparison of Repair Methods on SecVulEval and PrimeVul

Method	Base Model	Training Strategy	SecVulEval		PrimeVul	
			CodeBLEU	EM	CodeBLEU	EM
PCRepair	ChatGPT-5	Few-shot	**52.49**	**26.87**	**52.04**	**27.02**
PCRepair	ChatGPT-4	Few-shot	50.12	24.56	49.31	26.14
Vul-R2	Qwen-14B-Instruct-1M	SFT/RLVR	–	–	46.17	24.83
APPATCH	ChatGPT-4	COT	43.34	20.95	41.23	17.59
VulMaster	CodeT5	SFT	39.53	15.32	36.74	4.59
VulRepair	CodeT5	SFT	32.86	8.74	32.86	1.84
ChatGPT	ChatGPT-5	None	29.14	11.65	30.57	14.73
ChatGPT	ChatGPT-4	None	27.07	9.79	25.18	13.46

to the base ChatGPT-4 and ChatGPT-5, PCRepair also achieves consistent improvements across both CodeBLEU and EM. For example, under ChatGPT-5 on SecVulEval, CodeBLEU improves from 29.14 to 52.49 and EM from 11.65 to 26.87.

A similar trend is observed on the PrimeVul benchmark, where PCRepair achieves the highest overall performance, demonstrating advantages over recent SOTA LLM-based methods, Vul-R2 and APPATCH. First, PCRepair outperforms Vul-R2—which relies on extensive supervised fine-tuning and reinforcement learning (SFT & RLVR)—by achieving a CodeBLEU of 52.04 and an EM of 27.02, compared to Vul-R2's 46.17 and 24.83. Second, to ensure a fair comparison, we evaluate both PCRepair and APPATCH using ChatGPT-4. Under this setting, PCRepair significantly surpasses APPATCH, improving CodeBLEU from 41.23 to 49.31 and EM from 17.59 to 26.14. Beyond these SOTA approaches, PCRepair also maintains a clear superiority over learning-based models like VulMaster, which achieves a CodeBLEU of 36.74 and an EM of only 4.59. Finally, compared to the ChatGPT baselines, PCRepair yields massive improvements (e.g., increasing CodeBLEU from 30.57 to 52.04 and EM from 14.73 to 27.02 on ChatGPT-5).

*5.2. **RQ2: Impact of Core Design Choices***

Setup. This research question aims to evaluate the importance of the core design in our approach, premise chain matching. To this end, we conduct a comparative study using five alternative matching strategies:

(1) **Random Selection** serves as a lower-bound baseline. For each vulnerable instance, we only consider historical fixes from the same CWE category. We then randomly sample two distinct historical fixes by uniformly

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

selecting their indices without replacement. This strategy does not exploit any structural or semantic information from the vulnerable code.

(2) **Control Flow Matching** selects reference examples based on similarity in control flow. We leverage Joern [14] to generate control flow graphs (CFGs) for both the vulnerable function and candidate historical fixed vulnerabilities. Guided by fault localization, we perform backward traversal from the vulnerable statements to extract code patterns related to control flow. We then compute similarity based on the extracted CFG structures and select the top two most relevant historical fixed vulnerabilities as references.

(3) **Coarse Filtering** relies solely on the coarse matching stage of PCRepair. Specifically, we compute similarity scores between the vulnerable code and all candidates in the knowledge pool and directly select the top-2 examples. In contrast to PCRepair, this strategy omits the premise chain matching stage.

(4) **CodeSage Matching** uses CodeSage [30], a recent code representation model, as an embedding-based retrieval baseline. For each vulnerable instance and each candidate historical fix, we encode the corresponding vulnerable function into dense vector representations using CodeSage. We then compute cosine similarity between the target function and each candidate, and select the top-2 most similar historical fixes as references.

(5) **UniXcoder Matching** uses UniXcoder [13], a widely used pretrained code representation model, as another embedding-based retrieval baseline. Similar to the CodeSage strategy, we encode the target vulnerable function and candidate historical fixes into vector representations, compute their cosine similarity, and select the top-2 candidates with the highest similarity scores.

To control evaluation cost, we uniformly sample 2% of vulnerabilities from each CWE category on SecVulEval, resulting in a total of 135 vulnerabilities for this experiment.

Results. Table 2 reports the repair performance of different matching strategies, measured by CodeBLEU and EM. Overall, PCRepair achieves the best performance across both metrics, with a CodeBLEU score of 52.35 and an EM score of 26.43. Among the alternative strategies, the control flow performs second best, achieving 43.07 in CodeBLEU and 15.14 in EM. The embedding-based retrieval baselines also achieve competitive results, with CodeSage obtaining 41.24 in CodeBLEU and 13.85 in EM, and UniXcoder obtaining 36.87 in CodeBLEU and 11.36 in EM. In contrast, the Coarse-only and Random strategies show lower performance, indicating that simple

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

Table 2: Comparison of Different Matching Strategies

	PCRepair	**Random**	**Control Flow**	**Coarse**	**CodeSage**	**UniXcoder**
CodeBLEU	**52.35**	22.93	43.07	38.86	41.24	36.87
EM	**26.43**	8.39	15.14	9.76	13.85	11.36

Table 3: Premise chain quality and repair performance under different LLMs.

Model	Abstraction		Causality		Total	Total (Normalized)	CodeBLEU
	Logical Completeness	Semantic Accuracy	Dependency Correctness	Completeness of Premises			
ChatGPT-5	1.47	1.84	1.92	1.96	7.19	0.898	52.48
Qwen-3-8B	1.06	1.67	1.86	1.82	6.41	0.801	43.67
Llama-3.1-8B-Instruct	0.89	1.53	1.72	1.82	5.96	0.745	38.65

coarse matching or random selection is insufficient for retrieving effective repair references.

These results show that PCRepair outperforms not only random selection, coarse filtering, and control flow matching, but also representative embedding-based code retrieval methods. This suggests that directly comparing vulnerable functions through general-purpose code embeddings may not be sufficient to capture repair-relevant similarity. By reformulating vulnerable functions into premise chain, PCRepair captures vulnerability-relevant logic and dependency relations more explicitly, enabling more effective retrieval of historical fixes for patch generation.

5.3. RQ3: Impact of premise chain Construction Quality on Vulnerability Repair

Evaluating the quality of the premise chain is inherently challenging due to the lack of ground truth. To address this, we design a scoring rubric from two complementary perspectives: abstraction and causality. For each vulnerability, we generated three distinct premise chains using three different LLMs: Qwen-3-8B, ChatGPT-5, and Llama-3.1-8B-Instruct. We then assessed the quality of these generated chains using the proposed rubric.

In addition to the premise chain quality scores, we also report the corresponding downstream repair performance measured by CodeBLEU. This allows us to examine whether higher-quality premise chain representations are associated with better patch generation performance.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

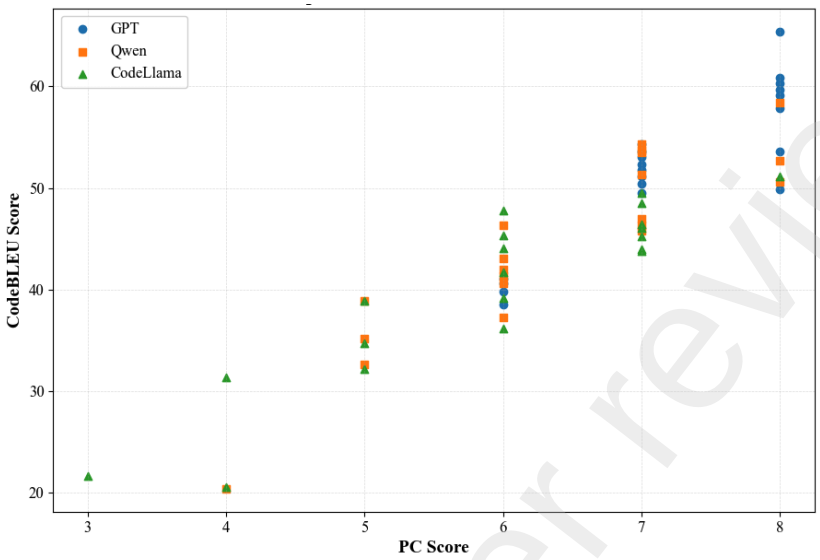


Figure 4: Relationship between premise chain Quality and Repair Performance for Each Vulnerability

Abstraction (0–4 per block). Abstraction evaluates whether each block forms a clear and semantically accurate logical step.

A1. Logical Completeness (0–2). Score 2: The block represents a clear and complete logical step with a well-defined purpose, neither too fine-grained nor too coarse. Score 1: The block represents a logical step but is imperfectly scoped (slightly too fine-grained or too coarse). Score 0: The block does not form a clear and complete step (e.g., a syntax-level fragment or a combination of unrelated actions).

A2. Semantic Accuracy (0–2). Score 2: The description is accurate and consistent with the code, with no incorrect or missing information. Score 1: The description is partially accurate, with minor inaccuracies or imprecise wording. Score 0: The description is inaccurate, including hallucinated behavior, missing key logic, or misleading interpretation.

Causality (0–4 per block). Causality evaluates whether the dependencies between blocks are correct and complete.

C1. Dependency Correctness (0–2). Score 2: All premises are correct. Score 1: The premises are partially incorrect or unclear. Score 0: Most or all premises are incorrect.

C2. Completeness of Premises (0–2). Score 2: All necessary premises are

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

included. Score 1: The premises are partially missing. Score 0: Most or key premises are missing.

Scoring. Each block is scored as: Abstraction (0–4) + Causality (0–4), with a maximum of 8. The total premise chain quality score is normalized by dividing the average block-level score by the maximum score of 8.

Findings The results in Table 3 show a consistent relationship between premise chain quality and downstream repair performance. ChatGPT-5 achieves the highest overall premise chain quality score (7.19) and also obtains the best CodeBLEU score (52.48). Qwen-3-8B ranks second in both premise chain quality (6.41) and CodeBLEU (43.67), while Llama-3.1-8B-Instruct achieves the lowest premise chain quality score (5.96) and the lowest CodeBLEU score (38.65). This trend suggests that higher-quality premise chain construction is associated with better downstream repair performance.

Fig. 4 further visualizes this relationship at the vulnerability level. Each point represents one vulnerability, with the x-axis showing its premise chain score and the y-axis showing the corresponding CodeBLEU score after patch generation. The points show a clear upward trend: vulnerabilities with higher premise chain scores generally achieve higher CodeBLEU scores. In particular, ChatGPT-5 produces more points in the high-premise chain score and high-CodeBLEU region, while Qwen-3-8B and Llama-3.1-8B-Instruct have more points in the lower-score regions.

A closer look at the rubric scores shows that the main gap among models lies in abstraction rather than causality. In particular, Logical Completeness is the largest bottleneck. ChatGPT-5 achieves the highest score (1.47), whereas Llama-3.1-8B-Instruct scores the lowest (0.89). This indicates that smaller models are more likely to produce poorly scoped blocks, either by fragmenting code into small syntax-level pieces or by merging unrelated actions into one block. Such decomposition errors directly weaken the usefulness of premise chain, because subsequent matching relies on these blocks as the basic units for comparing repair logic. When the blocks do not capture complete and meaningful logical actions, premise chain matching may align the wrong code patterns and retrieve less relevant historical vulnerabilities. This is reflected in the downstream repair results: Llama-3.1-8B-Instruct, which has the lowest abstraction quality, also obtains the lowest CodeBLEU score (38.65), while ChatGPT-5, which produces the highest-quality premise chain, achieves the best CodeBLEU score (52.48). The gap in Semantic Accuracy follows a similar pattern, with ChatGPT-5 achieving a higher score than Llama-3.1-8B-Instruct (1.84 vs. 1.53). This means that ChatGPT-5

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

Table 4: Performance under different parameter settings

Setting	Variant	CodeBLEU	Cost
Premise Chain Ratio	3%	44.46	0.04
	5%	49.28	0.07
	10%	**52.54**	0.14
	15%	52.84	0.21
Reference Fixes (Top-k)	Top-1	48.46	–
	Top-2	**52.49**	–
	Top-3	48.75	–

describes the code behavior more accurately and produces fewer hallucinations.

In contrast, the gap is much smaller in the Causality scores. For both Dependency Correctness and Completeness of Premises, all models score relatively high, with scores above 1.72. This indicates that even smaller models can often identify how different code blocks depend on each other. In other words, the main weakness of smaller models is not dependency identification, but block decomposition. If a model fails to divide the code into complete and accurate logical blocks at the beginning, these early errors will carry over to the subsequent premise chain matching process. Therefore, high-quality premise chain construction is important for retrieving useful repair examples and improving patch generation performance.

5.4. ***RQ4: Hyper-parameter Selection***

Setup. In this research question, we study how the choice of hyper-parameters affects the performance of PCRepair. We focus on two parameters that directly influence the matching and generation stages: (1) the proportion of candidate vulnerabilities selected for premise chain matching, for which we consider four settings—3%, 5%, 10%, and 15%. This parameter controls the trade-off between matching cost and the diversity of candidate examples; and (2) the number of reference examples used to guide patch generation, which we fix to the top 2 most relevant historical fixes identified by our matching strategy.

To evaluate the impact of these design choices, we conduct a controlled analysis under different hyper-parameter configurations and compare their

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

effects on overall repair effectiveness. For this study, we use a subset of 135 vulnerabilities from SecVulEval following RQ2.

Results. Based on the results reported in Table 4, several important insights can be derived regarding the selection of the premise chain ratio and the number of reference fixes.

- For the premise chain ratio, the matching performance improves steadily as more candidates are retained in the cost-aware coarse filtering stage. Specifically, increasing the premise chain ratio from 3% to 10% leads to a substantial CodeBLEU improvement, from 44.46 to 52.54. This trend indicates that keeping a larger portion of relevant candidates effectively enriches the information available for subsequent premise chain matching. However, further increasing the premise chain ratio to 15% yields only a marginal improvement in CodeBLEU, from 52.54 to 52.84, while the computational cost continues to increase significantly, from 0.14 to 0.21. This clear saturation effect suggests that although broader historical fixes can be beneficial, the additional gains beyond the 10% premise chain ratio are limited and do not justify the increased overhead. Overall, these results highlight that the 10% premise chain ratio represents a well-balanced operating point, achieving strong performance with moderate cost.

- In terms of the number of reference fixes, we observe a non-linear relationship between the amount of guidance provided and the final generation quality. Using only the most relevant fix (Top-1) results in a moderate CodeBLEU score of 48.46, indicating that a single example may not provide sufficient diversity for robust repair. Expanding the reference set to include the two most relevant historical fixes (Top-2) significantly improves performance to 52.49, demonstrating the benefit of incorporating complementary repair patterns. However, further increasing the number of references to three (Top-3) leads to a noticeable performance drop (48.75). This decline suggests that while moderate diversity is helpful, introducing too many examples can dilute the relevance of guidance and introduce noise into the generation process.

6. Discussion

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

6.1. Failure Case Analysis

We conduct a failure case analysis of PCRepair based on the 135 vulnerabilities studied in RQ2 to investigate common failure patterns and their underlying causes. Since one vulnerability may exhibit multiple failure symptoms, the percentages reported below are not mutually exclusive. We observe three major symptom groups.

Low-quality premise chain construction. A common symptom is that the generated premise chain is of low quality (34.1%). This includes cases where the decomposition produces: (1) fragmented decomposition, where the generated blocks are poorly defined and fail to encapsulate clear and complete logical steps; (2) incorrect dependencies, where the relationships between blocks are incorrectly identified or omitted; and (3) inaccurate semantic descriptions, where the textual summaries of blocks do not faithfully reflect the underlying code behavior. Such issues lead to an unreliable representation of the program logic, which negatively affects subsequent stages.

We observed that the quality of the premise chain decomposition is sensitive to the reasoning and instruction-following capabilities of the underlying LLM. Advanced large-scale models, such as GPT-5, consistently exhibit the best performance in generating well-structured and accurate premise chain. In contrast, when employing smaller-scale open-source models, such as Qwen-2.5-7B-Instruct or Llama-3.1-8B-Instruct, the frequency of the aforementioned decomposition errors increases. Due to their limited parameter capacity, these models often struggle to fully comprehend complex, long-context code semantics for premise chain construction.

Ineffective retrieval of relevant fixes. Another failure pattern is ineffective retrieval. In 11.2% of the inspected cases, the target vulnerability arises from a specific or atypical scenario, such as an edge case. Although the retrieved examples may share the same CWE type, their vulnerable logic is not sufficiently aligned with the target, and thus the corresponding fixes provide limited guidance for patch generation. In addition, 2.7% of the cases involve CWE categories with only a small number of available candidate fixes. This limited candidate pool reduces the chance of retrieving useful historical examples, even when the matching strategy is effective.

Complex repairs requiring broader project context. We also find that 33.8% of the inspected cases require information beyond similar historical fixes. These cases often involve complex repair logic that depends on project-specific APIs, file-level context, or even the broader design intent of the project. For example, some fixes require understanding how internal

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

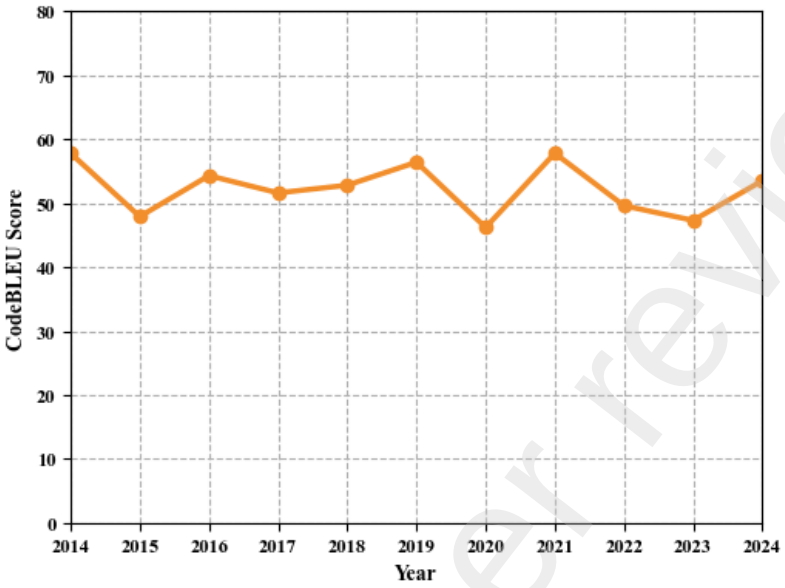


Figure 5: CodeBLEU Performance by Vulnerability Year

APIs should be used, how state is maintained across multiple functions, or what assumptions are enforced elsewhere in the project. In such cases, retrieving similar vulnerability-fix pairs alone may be insufficient, because the LLM also needs additional contextual information, such as documentation, API usage examples, or broader file/project-level semantics, to generate a correct patch.

6.2. Analysis of Data Leakage

To provide additional discussion on the potential effect of temporal leakage, we analyze ChatGPT performance across vulnerabilities from different disclosure years. Specifically, we group the evaluated vulnerabilities by disclosure year and compare the corresponding repair performance of our approach across these temporal subsets.

Following the setup described above, Fig. 5 reports the repair performance of our approach for vulnerabilities grouped by disclosure year. The dataset covers vulnerabilities from 2014 to 2024, with most samples concentrated in the later years. The average CodeBLEU score across all years is 50.08. Overall, the yearly results remain relatively stable, with no obvious trend showing that older vulnerabilities are consistently easier to repair than newer

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

ones. We also note that ChatGPT-4 was trained on data collected up to approximately 2023. However, in this analysis, we do not observe a clear and consistent performance gap between vulnerabilities disclosed before and after this approximate cutoff. This observation provides limited evidence that pre-training cutoff alone explains the performance of our approach, although it does not rule out the possibility of data leakage or memorization.

6.3. Threats to Validity

One potential threat to internal validity arises from the dependence of PCRepair on the quality of premise chain construction generated by large language models. Errors or inconsistencies in this decomposition may affect subsequent retrieval and alignment, thereby influencing repair performance. To mitigate this threat, we apply a consistent decomposition procedure across all experiments and validate sampled premise chain representations to ensure structural soundness. Another concern is the potential impact of data leakage, as some vulnerabilities or fixes may overlap with the pre-training data of the underlying LLMs. Although this risk cannot be fully eliminated, we reduce its influence by conducting comparative evaluations under identical settings, ensuring that performance gains are attributable to our retrieval and premise chain design rather than memorization.

The generalizability of our findings may be limited by the scope of datasets and programming languages considered. Our experiments focus on C/C++ vulnerabilities on SecVulEval and PrimeVul, which, although representative, may not cover the full diversity of real-world systems. While premise chain and the retrieval-based framework are designed to be language-agnostic in principle, further validation is needed to confirm their effectiveness across other languages and vulnerability domains. Extending PCRepair to additional datasets and software ecosystems remains an important direction for future work.

7. Related Work

Prior research on automated vulnerability repair (AVR) can be broadly categorized into four paradigms: template-guided, search-based, constraint-based, and learning-based methods. Template-guided approaches rely on predefined patterns; for example, AutoPaG [15] applies fixed instrumentation templates to mitigate out-of-bounds memory accesses, though such methods often lack flexibility for unseen vulnerabilities. Search-based methods explore

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

candidate patches via heuristic mutations, as seen in VulnFix [31], which employs snapshot fuzzing to infer patch invariants. Constraint-based techniques formulate repair by satisfying behavioral constraints, with ExtractFix [12] leveraging sanitizer reports to extract exact repair specifications. However, both search- and constraint-based methods typically require precise dynamic execution traces or exploits, limiting their broader applicability. To overcome these limitations, learning-based methods have gained prominence by extracting repair patterns directly from large-scale datasets. As a state-of-the-art representative, VulMaster [32] augments pre-trained code models with abstract syntax trees and CWE knowledge to effectively reason about and generate patches.

More recent work has explored vulnerability repair from a reasoning-oriented perspective. For example, Vul-R2 [27] views vulnerability repair from a reasoning perspective. It formulates repair as an exploratory-feedback problem, where LLMs iteratively reason over candidate fixes and use verifiable feedback to identify correct patches.

Similarly, APPATCH [19] leverages static analysis and LLMs to perform vulnerability slicing and root cause analysis, which are then used to guide patch generation. Compared with APPATCH, PCRepair differs in three key aspects. First, in code semantics representation, APPATCH relies on static analysis (dependency analysis) and execution order, while PCRepair uses a premise chain structure to capture logical dependencies beyond strict syntactic structures. Second, in context coverage, APPATCH applies backward slicing, which may miss relevant information outside the slice, whereas PCRepair models global semantic relations to include broader context. Third, in robustness, APPATCH depends on LLM-generated root causes, making it sensitive to analysis errors. In contrast, PCRepair avoids explicit root cause generation and retrieves relevant fixes based on semantic structure, leading to more stable performance.

Retrieval-based methods have been used to provide similar functions or historical fixes as guidance for repair. Many existing strategies rely on general-purpose code similarity, such as code embeddings produced by models like CodeSage [30] and UniXcoder [13]. These methods encode the target vulnerable function and candidate historical fixes into vector representations and retrieve candidates with high cosine similarity. Although such approaches can capture general code-level similarity, they are not specifically designed for vulnerability repair. They may retrieve functions that are syntactically or structurally similar but not aligned with the vulnerability-

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

relevant behavior or repair intent. In contrast, PCRepair is tailored for vulnerability-fix retrieval by using the premise chain representation to model vulnerability-relevant semantic blocks and dependency relations, thereby retrieving historical fixes that better reflect the repair logic needed for AVR.

8. Conclusion

In this paper, we introduced PCRepair, an AVR framework that reformulates vulnerable functions into a novel code representation called premise chain and leverages it to retrieve relevant historical fixes for LLM-based vulnerability repair. The premise chain representation organizes program logic into semantically coherent blocks connected through explicit dependency relations, while filtering out irrelevant code patterns. By incorporating the matched relevant historical fixes as references, PCRepair guides LLM-based vulnerability repair.

We evaluated PCRepair on two widely used C/C++ benchmarks, SecVulEval and PrimeVul, using CodeBLEU and EM. Overall, PCRepair demonstrates consistently strong results across both datasets on CodeBLEU and Exact Match. On SecVulEval, PCRepair achieves a CodeBLEU score of 52.49, a 32.8% relative increase over the best baseline; on PrimeVul, the score is 52.04, reflecting an 11.5% increase. These results indicate that premise chain representations capture vulnerability-relevant code and semantics, enable effective retrieval of relevant historical fixes, and guide LLM-based vulnerability repair.

9. Data Availability

To facilitate the replication and future research, we have made our implementation and dataset available [2].

References

[1] Ahmed, M.B.U., Harzevili, N.S., Shin, J., Pham, H.V., Wang, S., 2025. Secvuleval: Benchmarking llms for real-world c/c++ vulnerability detection. arXiv preprint arXiv:2505.19828 .

[2] Anonymous, 2026. Pcrepair: Anonymous repository. `https://github.com/Anonymousllea/PCRepair.git`.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

[3] Bhandari, G., Naseer, A., Moonen, L., 2021. Cvefixes: automated collection of vulnerabilities and their fixes from open-source software, in: Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering, pp. 30–39.

[4] Chen, Y., Ding, Z., Alowain, L., Chen, X., Wagner, D., 2023. Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection, in: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses, pp. 654–668.

[5] Chen, Z., Kommrusch, S., Monperrus, M., 2022. Neural transfer learning for repairing security vulnerabilities in c code. IEEE TSE 49, 147–165.

[6] Croft, R., Babar, M.A., Kholoosi, M.M., 2023. Data quality for software vulnerability datasets, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE. pp. 121–133.

[7] Ding, Y., Fu, Y., Ibrahim, O., Sitawarin, C., Chen, X., Alomair, B., Wagner, D., Ray, B., Chen, Y., 2024. Vulnerability detection with code language models: How far are we? arXiv preprint arXiv:2403.18624 .

[8] Edgescan, 2022. January 2022 vulnerability statistics snapshot. URL: `https://www.edgescan.com/january-2022-vulnerability-statistics-snapshot/`. accessed: 26 Jan 2026.

[9] Fan, J., Li, Y., Wang, S., Nguyen, T.N., 2020. Ac/c++ code vulnerability dataset with code changes and cve summaries, in: Proceedings of the 17th international conference on mining software repositories, pp. 508–512.

[10] Forsgren, N., Alberts, B., Backhouse, K., Baker, G., Cecarelli, G., Jedamski, D., Kelly, S., Sullivan, C., 2021. 2020 state of the octoverse: Securing the world's software. arXiv preprint arXiv:2110.10246 .

[11] Fu, M., Tantithamthavorn, C., Le, T., Nguyen, V., Phung, D., 2022. Vulrepair: a t5-based automated software vulnerability repair, in: Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering, pp. 935–947.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

[12] Gao, X., Wang, B., Duck, G.J., Ji, R., Xiong, Y., Roychoudhury, A., 2021. Beyond tests: Program vulnerability repair via crash constraint extraction. ACM Transactions on Software Engineering and Methodology (TOSEM) 30, 1–27.

[13] Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., Yin, J., 2022. Unixcoder: Unified cross-modal pre-training for code representation, in: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 7212–7225.

[14] Joern Project, 2024. Joern: The bug hunter's workbench. URL: `https://joern.io/`. accessed: 26 Jan 2026.

[15] Lin, Z., Jiang, X., Xu, D., Mao, B., Xie, L., 2007. Autopag: towards automated software patch generation with source code root cause identification and repair, in: Proceedings of the 2nd ACM symposium on Information, computer and communications security, pp. 329–340.

[16] Mell, P., Scarfone, K., Romanosky, S., 2007. The Common Vulnerability Scoring System (CVSS) and Its Applicability to Federal Agency Systems. NIST Interagency Report 7435. National Institute of Standards and Technology (NIST). URL: `https://nvlpubs.nist.gov/nistpubs/legacy/ir/nistir7435.pdf`. published August 2007; accessed 26 Jan 2026.

[17] Mukherjee, S., Chinta, A., Kim, T., Sharma, T.A., Hakkani-Tür, D., 2025. Premise-augmented reasoning chains improve error identification in math reasoning with llms. arXiv preprint arXiv:2502.02362 .

[18] Nikitopoulos, G., Dritsa, K., Louridas, P., Mitropoulos, D., 2021. Crossvul: a cross-language vulnerability dataset with commit data, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 1565–1569.

[19] Nong, Y., Yang, H., Cheng, L., Hu, H., Cai, H., 2025. {APPATCH}: Automated adaptive prompting large language models for {Real-World} software vulnerability patching, in: 34th USENIX Security Symposium (USENIX Security 25), pp. 4481–4500.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

[20] OpenAI, 2023. Gpt-4. https://openai.com/index/gpt-4-research/. OpenAI research page, accessed April 13, 2026.

[21] OpenAI, 2025. Introducing gpt-5. URL: https://openai.com/index/introducing-gpt-5/. accessed: 26 Jan 2026.

[22] Papineni, K., Roukos, S., Ward, T., Zhu, W.J., 2002. Bleu: a method for automatic evaluation of machine translation, in: Proceedings of the 40th annual meeting of the Association for Computational Linguistics, pp. 311–318.

[23] Python Software Foundation, 2026. Welcome to python.org. URL: https://www.python.org/. accessed: 26 Jan 2026.

[24] Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., Sundaresan, N., Zhou, M., Blanco, A., Ma, S., 2020. Codebleu: a method for automatic evaluation of code synthesis. arXiv preprint arXiv:2009.10297 .

[25] Statista, 2024. Number of common it security vulnerabilities and exposures (cves) worldwide from 2009 to 2024 ytd. URL: https://www.statista.com/statistics/500755/worldwide-common-vulnerabilities-and-exposures/. accessed: 26 Jan 2026.

[26] Wen, X.C., Gao, C., Luo, F., Wang, H., Li, G., Liao, Q., 2024. Livable: exploring long-tailed classification of software vulnerability types. IEEE Transactions on Software Engineering 50, 1325–1339.

[27] Wen, X.C., Lin, Z., Yang, Y., Gao, C., Ye, D., 2025. Vul-r2: A reasoning llm for automated vulnerability repair. arXiv preprint arXiv:2510.05480 .

[28] Wen, X.C., Wang, X., Gao, C., Wang, S., Liu, Y., Gu, Z., 2023. When less is enough: Positive and unlabeled learning model for vulnerability detection, in: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE. pp. 345–357.

[29] Xia, C.S., Zhang, L., 2023. Keep the conversation going: Fixing 162 out of 337 bugs for $0.42 each using chatgpt. arXiv preprint arXiv:2304.00385 .

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622

[30] Zhang, D., Ahmad, W., Tan, M., Ding, H., Nallapati, R., Roth, D., Ma, X., Xiang, B., 2024. Code representation learning at scale. arXiv preprint arXiv:2402.01935 .

[31] Zhang, Y., Gao, X., Duck, G.J., Roychoudhury, A., 2022. Program vulnerability repair via inductive inference, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 691–702.

[32] Zhou, X., Kim, K., Xu, B., Han, D., Lo, D., 2024. Out of sight, out of mind: Better automatic vulnerability repair by broadening input ranges and sources, in: Proceedings of the IEEE/ACM 46th international conference on software engineering, pp. 1–13.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6908622